

Lab Project Kick-off

Alessandro Bocci

name.surname@unipi.it

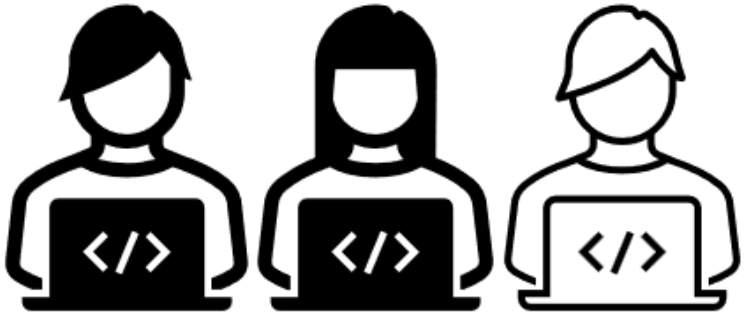
Advanced Software Engineering (Lab)

07/10/2026

The lab project



- Teams of 3-4 people.
- Deadline 15 December.
- Delivery:
  - Git repository with code and documentation.
  - Short report based on a template we will give you.
  - Discussion of the project as lab oral.



Software characteristics



Distributed application.



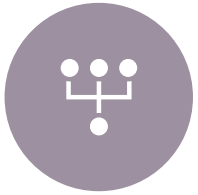
REST interface.



One or more databases.



Security mechanisms.

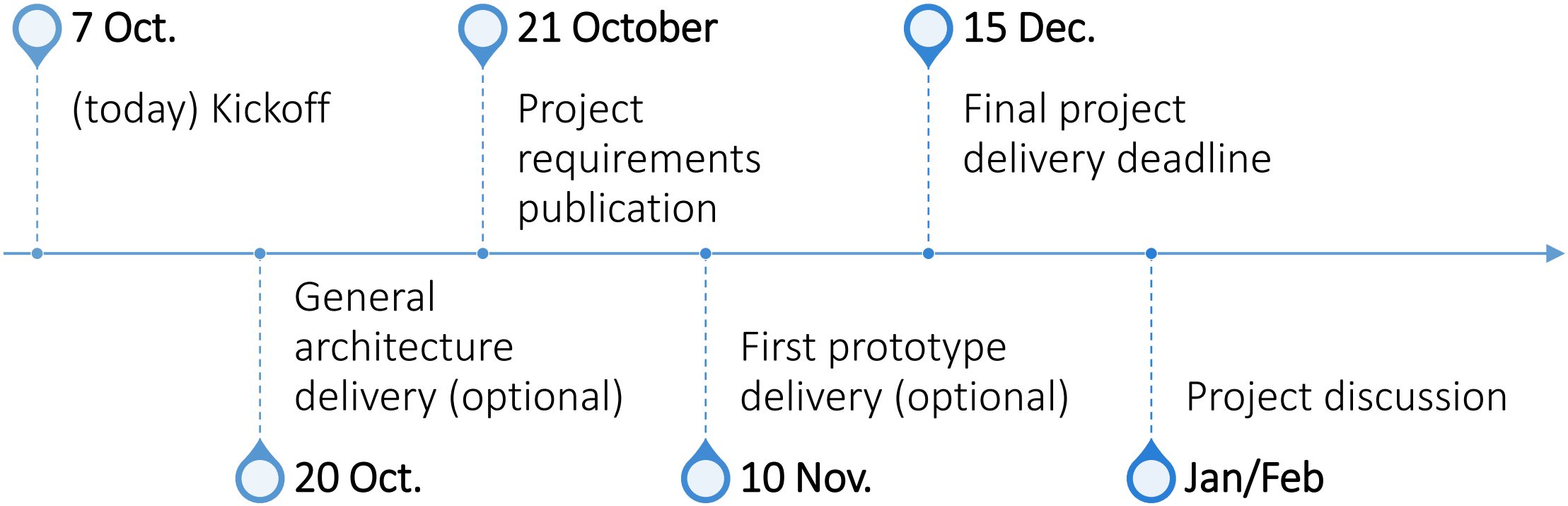


Unit and performance testing.



Security analyses.

Roadmap


7 Oct.
(today) Kickoff
21 October
Project requirements publication
15 Dec.
Final project delivery deadline
General architecture delivery (optional)
20 Oct.
First prototype delivery (optional)
10 Nov.
Project discussion
Jan/Feb

Why optional deliveries?

Optional = feedback

- No grade
- No penalty if you do not deliver
- Opportunity to catch design problems early



Earlier feedback is usually cheaper than later refactoring.

Initial design delivery

Design delivery, a sketch of a monolithic software architecture with:



THE MAIN COMPONENTS.



THE COMMUNICATION WITH THE USERS.



THE DATABASE(S) CONTENT.



THE COMPLETE USER STORIES.

First prototype delivery

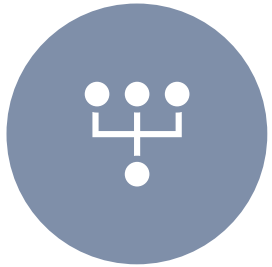
GitHub repository with:



- GENERAL DESCRIPTION



- CODE



- QUICK START (INSTALLATION AND EXECUTION)



- FULL API AND ITS TESTS

First prototype delivery

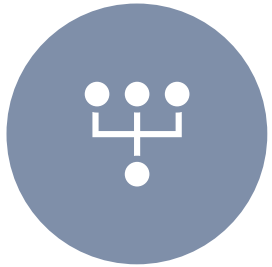
GitHub repository with:



- GENERAL DESCRIPTION



- CODE



- QUICK START (INSTALLATION AND EXECUTION)



- FULL API AND ITS TESTS

The next few labs will have details on how to do them

Final project delivery

GitHub repository as before.

Report describing the project.

I will give you a template for it.

The difference between the first prototype and the final delivery consists in some features (requirements) that will be covered later in the course

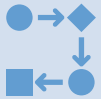
Project Discussion



The final delivery will receive the same grade for all members of a group.



During the exam period in January and February, I will provide several dates when your group can meet with me to discuss the project.



During the discussion, I will ask questions about the project's realization to assess and evaluate your individual involvement.



This discussion will add or subtract a score from the final delivery grade (applied individually, not shared among group members).



If a student is unable to answer questions and demonstrate sufficient involvement in the project, their final grade will be **"Failed"**, and they will have to take the exam without the project.

The lab groups



3 OR 4 PEOPLE.



USE THE SHARED DOCUMENT ON THE MOODLE TO FORM GROUPS.



DEADLINE FOR THE GROUP CREATION: NOVEMBER 1ST.

The project – Monster Tamers

Presented in Lab 2





The user stories

I merged the user stories produced last Lab (file on the Moodle).

The priority is given to indicate the minimal viable product for the project, not for a real product:

- Red: first delivery
- Yellow: final delivery
- Green: optional features suggested by you

The user stories

I tried to have general user stories to give you freedom and diversify the projects.

For example, there is no user story about encounters implementation.

All the specific ones are green so you can implement them or not.

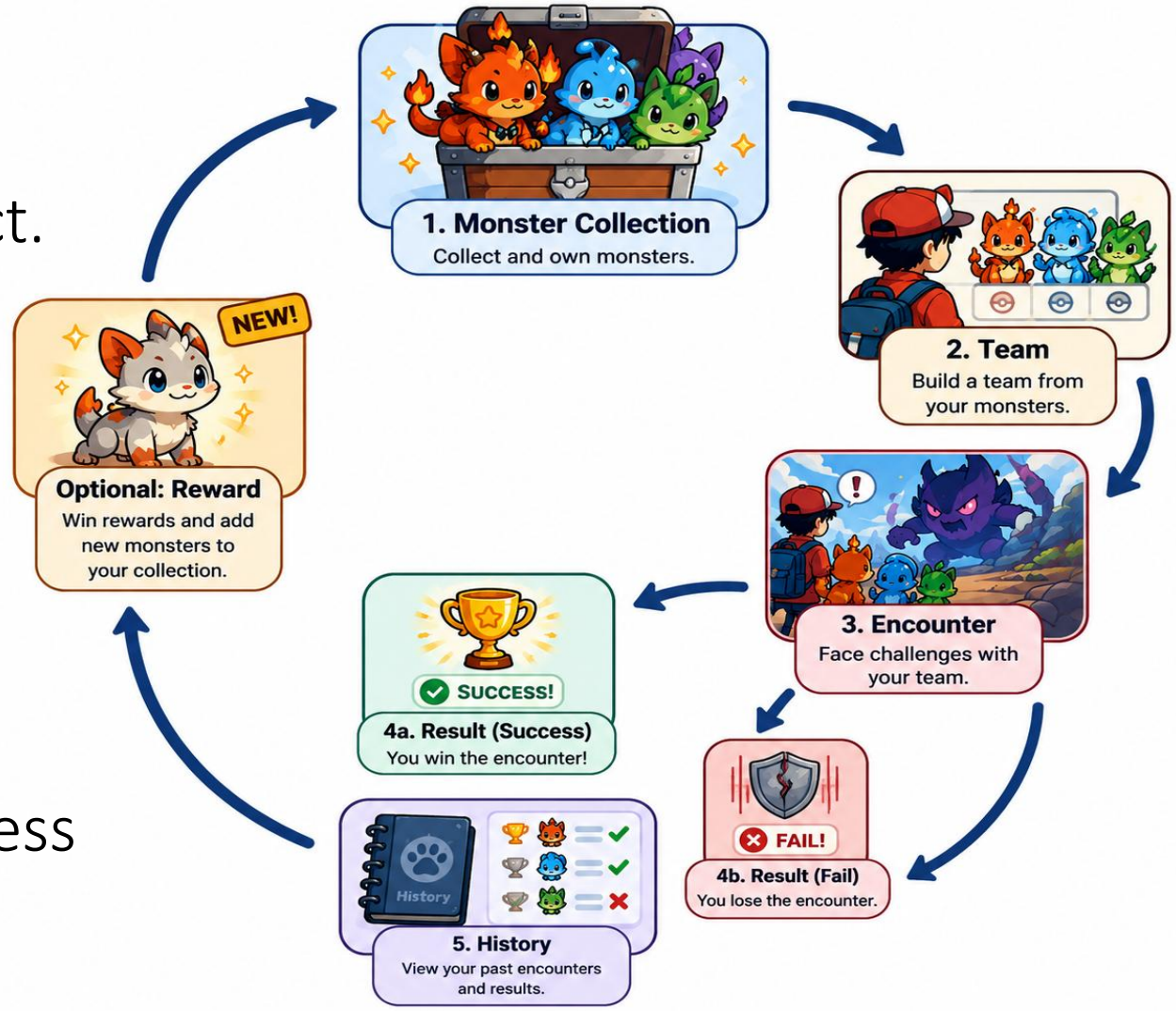
You can add also features that are not in the user stories.

The game loop

This is the minimal needed by the project.

You can extend it if you want.

Prepare → encounter → result → progress



The players

The information to represent a player is free for you.

You have to put at least:

- Some credentials
- Encounters history

You are free to add things like picture, friends etc.

You are free to separate profile and account of a player.

Monsters

- Every monster has its own characteristics
- Characteristics influence encounters
- Every player owns a personal collection
- New monsters may be obtained during the game



Examples only!!!

Monsters

Each group choose the monster collection of their project.

The monsters must have at least:

- An image representing them.
- Two characteristics for the gameplay of the encounters.

(You can add every characteristics you want)

Teams

Before an encounter, a player prepares a team using monsters from their collection. A team must:

- Have limited size
- Satisfy the game rules
- Be composed only by owned monsters

The exact team rules are up to you.

Encounters

An encounter is a challenge faced by a team with a ‘wild monster’.

- Different encounters may have different characteristics
- The result depends on the selected team
- An encounter may succeed or fail
- Some outcomes may grant a new monster



An encounter does **not** have to be a turn-by-turn battle.

Rules

You are mainly free to specify the rules of your game.

For example:

- The main game objective
- How increase the collection
- The encounters
- Duplicates in personal collections
- …

For instance: you can have fight-based encounters and the objective is to beat the final boss after several battles with increasing difficulty.

The backend

The service must manage:

- Overall and personal Monster collections
- Team selection
- Encounters
- History of encounters (with permanent storage)

Everything should be doable trough a RESTful interface.

No relevant business logic in a frontend!

The frontend

The main focus of the project is on the backend.

As a minimal debug system, I ask you to implement a minimal frontend with a basic UI that should use the backend's REST API for the basic operation.

The aspect is up to you, what is interesting for me is the usage and how it interacts with the backend.

No important business logic should be in the frontend.

Miscellaneous

There are some features we haven't explicitly mentioned yet:

- Programming languages.
- Database(s).
- …

You are mostly free about them.

After the first optional delivery a document having the requirements of the project will be published.

The features to be implemented are specified in the user stories.



Groups creation

In the course Moodle there is a shared document to support the group creation.

Follow the instruction on the first sheet to:

- Look for people to create a group.
- ‘Enroll’ your group.
- November 1st the file edit will be blocked.

Complete the user stories

In the file with the merged user stories I only put them with a priority.

You should define the acceptance criteria (at least) of the high and medium priority ones.

You should define the dependencies.



Example: the architecture

Simple and rough example of architecture of an online shop.

The user access the orders to:

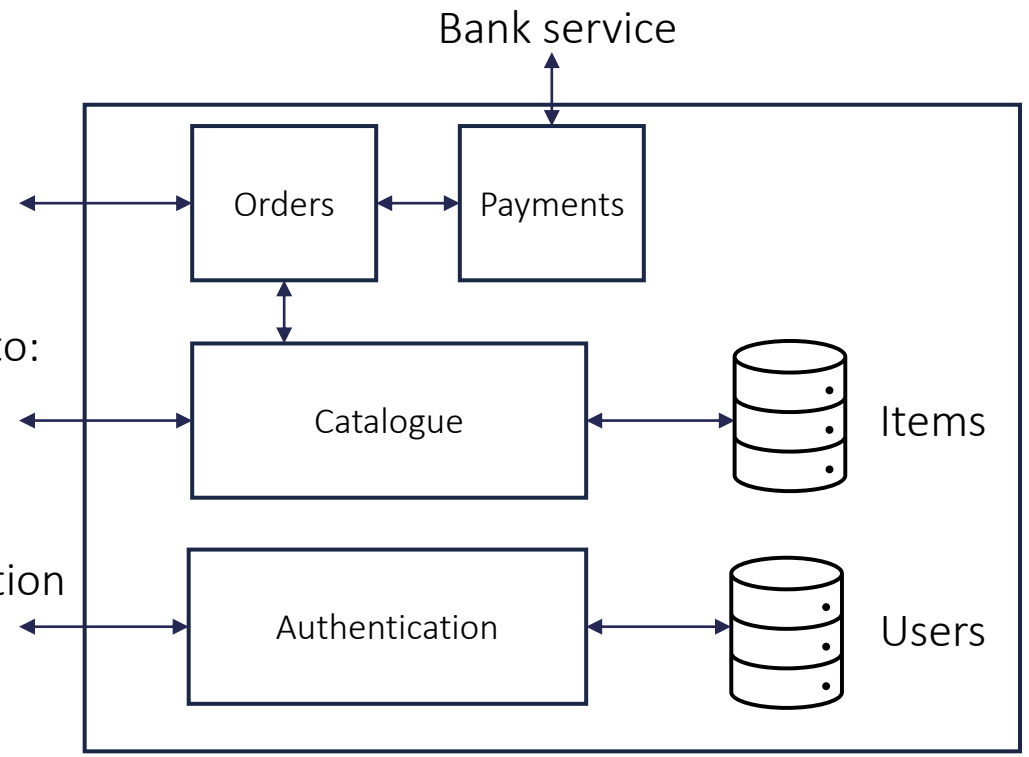
- Create new orders and pay
- Check the status

The user access the catalogue to:

- Check the shop items

The user access the authentication to:

- Make the login



Items records

- Name
- Price
- Stock
- …

Users records

- Username
- Password
- Invoice info
- History
- …

Example: the architecture

Simple and rough example of architecture of an online shop.

The user access the orders to:

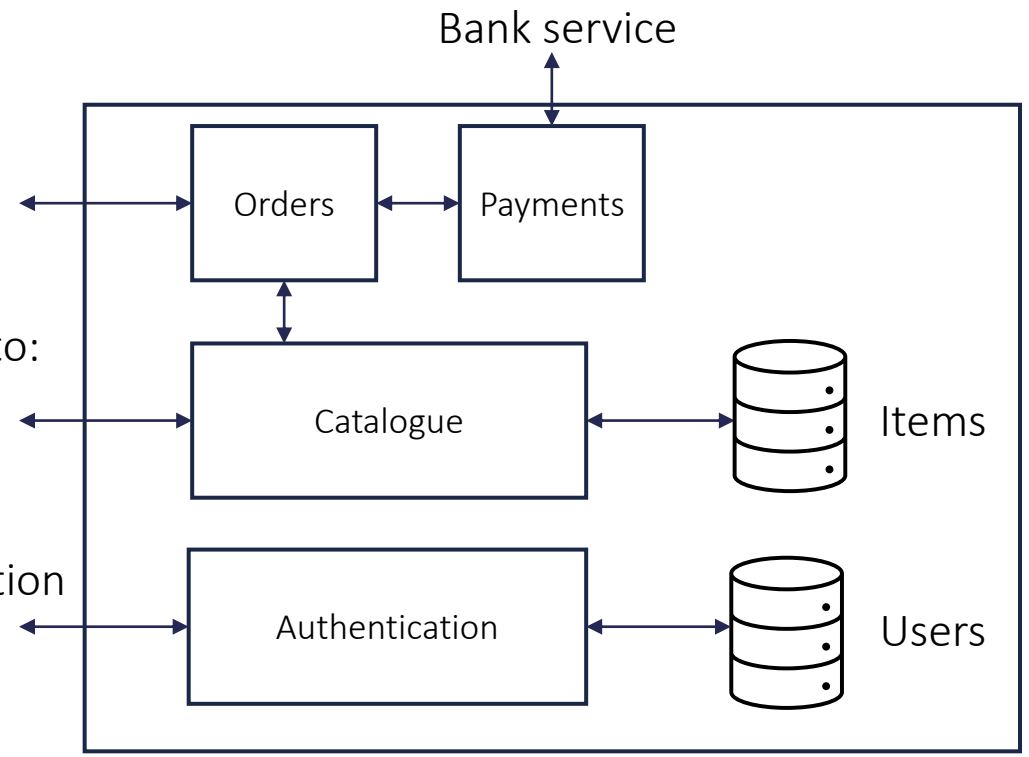
- Create new orders and pay
- Check the status

The user access the catalogue to:

- Check the shop items

The user access the authentication to:

- Make the login



Items records

- Name
- Price
- Stock
- …

Users records

- Username
- Password
- Invoice info
- History
- …

Example: the architecture

Simple and rough example of architecture of an online shop.

We don't have a DB for orders!

The user access the orders to:

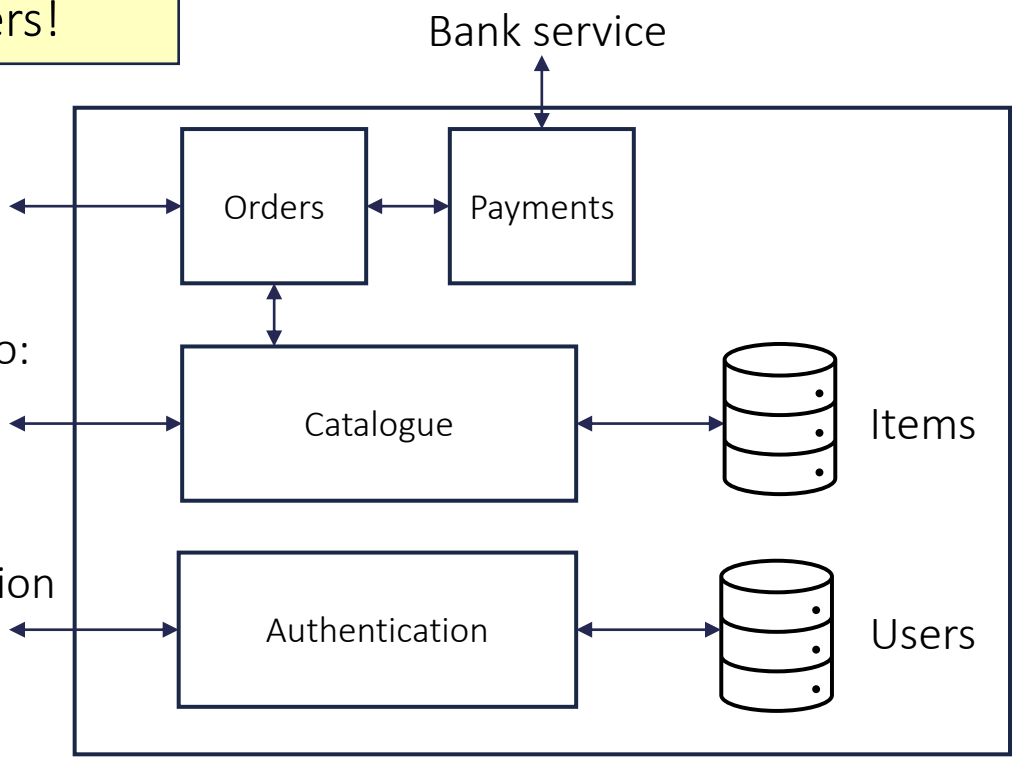
- Create new orders and pay
- Check the status

The user access the catalogue to:

- Check the shop items

The user access the authentication to:

- Make the login



Items records

- Name
- Price
- Stock
- …

Users records

- Username
- Password
- Invoice info
- History
- …

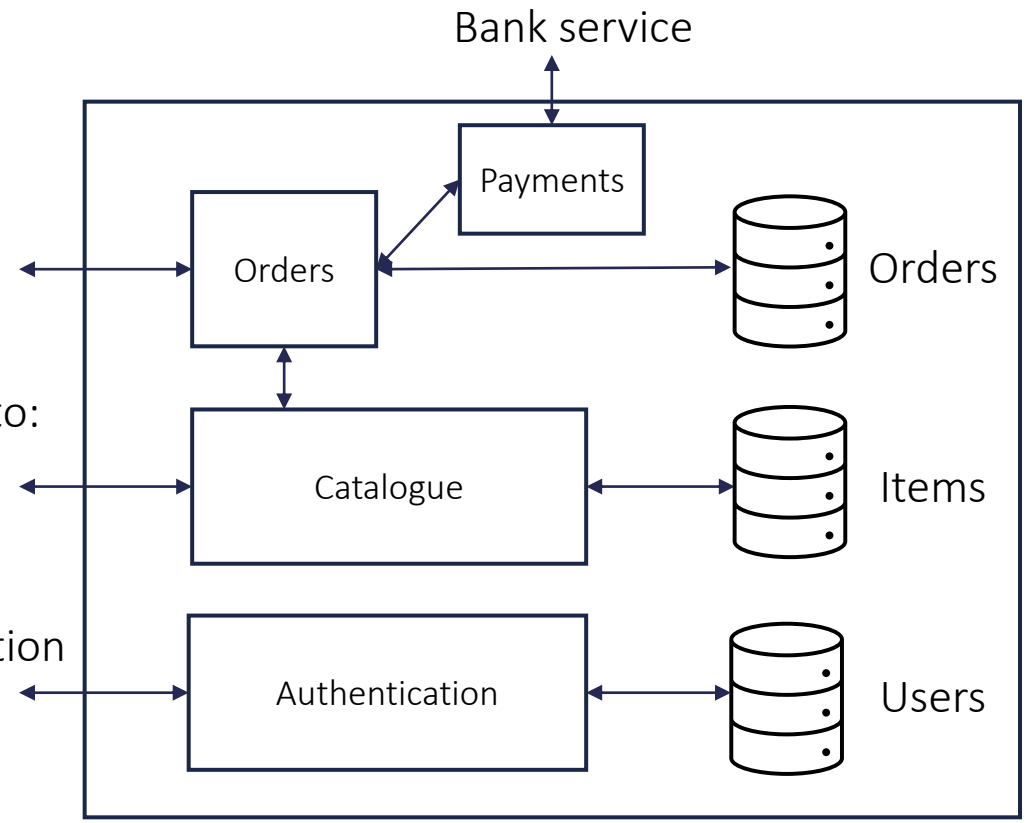
Example: the architecture

Simple and rough example of architecture of an online shop.

The user access the orders to:
- Create new orders and pay
- Check the status

The user access the catalogue to:
- Check the shop items

The user access the authentication to:
- Make the login



Orders records
- …

Items records
- Name
- Price
- Stock
- …

Users records
- Username
- Password
- Invoice info
- History
- …

Example: the architecture

Simple and rough example of architecture of an online shop.

The user access the orders to:

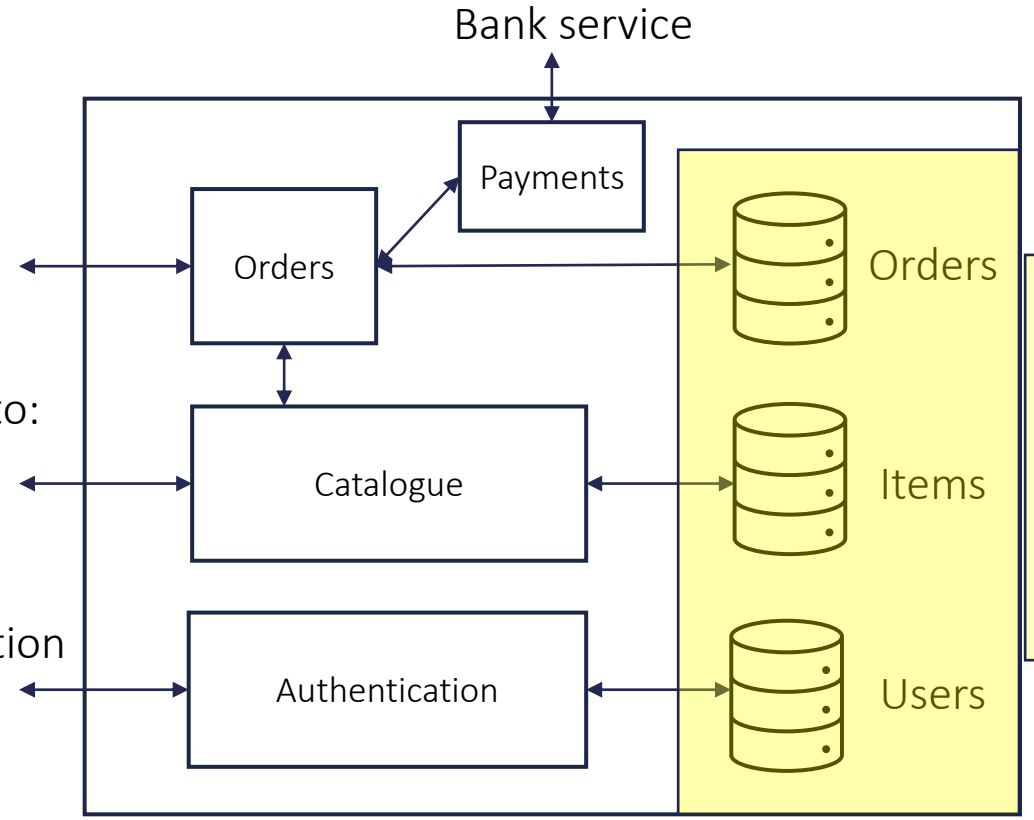
- Create new orders and pay
- Check the status

The user access the catalogue to:

- Check the shop items

The user access the authentication to:

- Make the login



They can be:

- Different DBMSs
- Different DB in the same DBMS
- Different tables in the same DB

It is up to you ☺

Example: the architecture

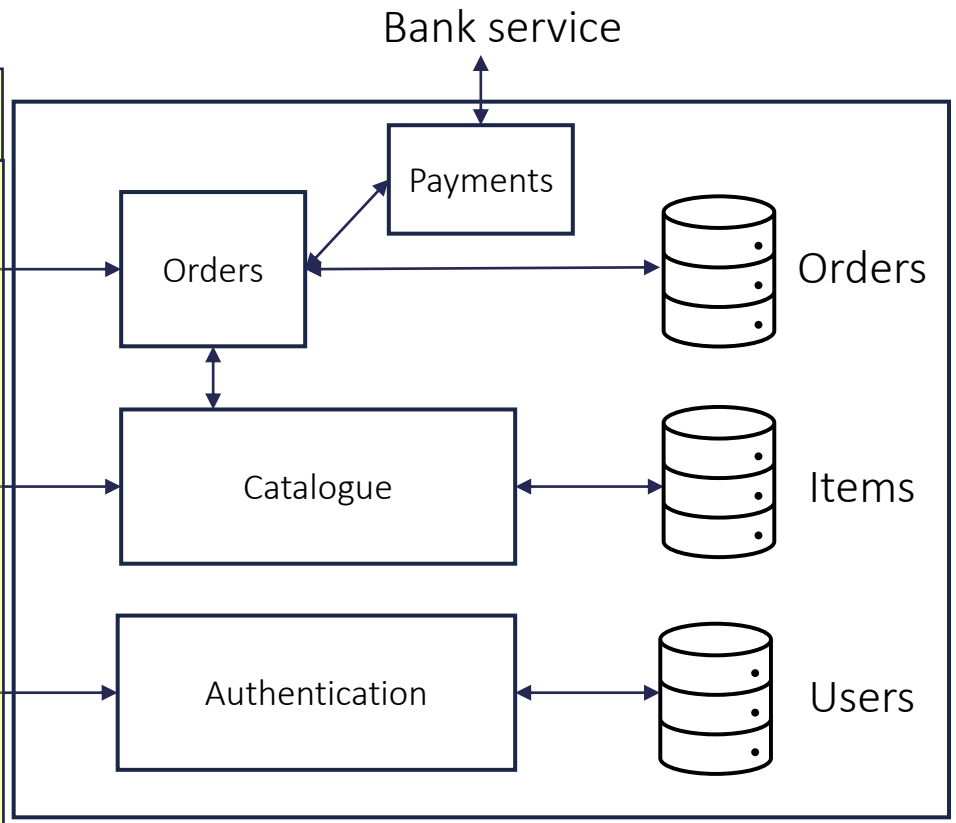
Simple and rough example of architecture of an online shop.

This must be expressed as a REST API

The user access the orders to:
- Create new orders and pay
- Check the status

The user access the catalogue to:
- Check the shop items

The user access the authentication to:
- Make the login



Orders records
- …

Items records
- Name
- Price
- Stock
- …

Users records
- Username
- Password
- Invoice info
- History
- …

Example: the architecture

Simple and rough example of architecture of an online shop.

The user access the orders to:

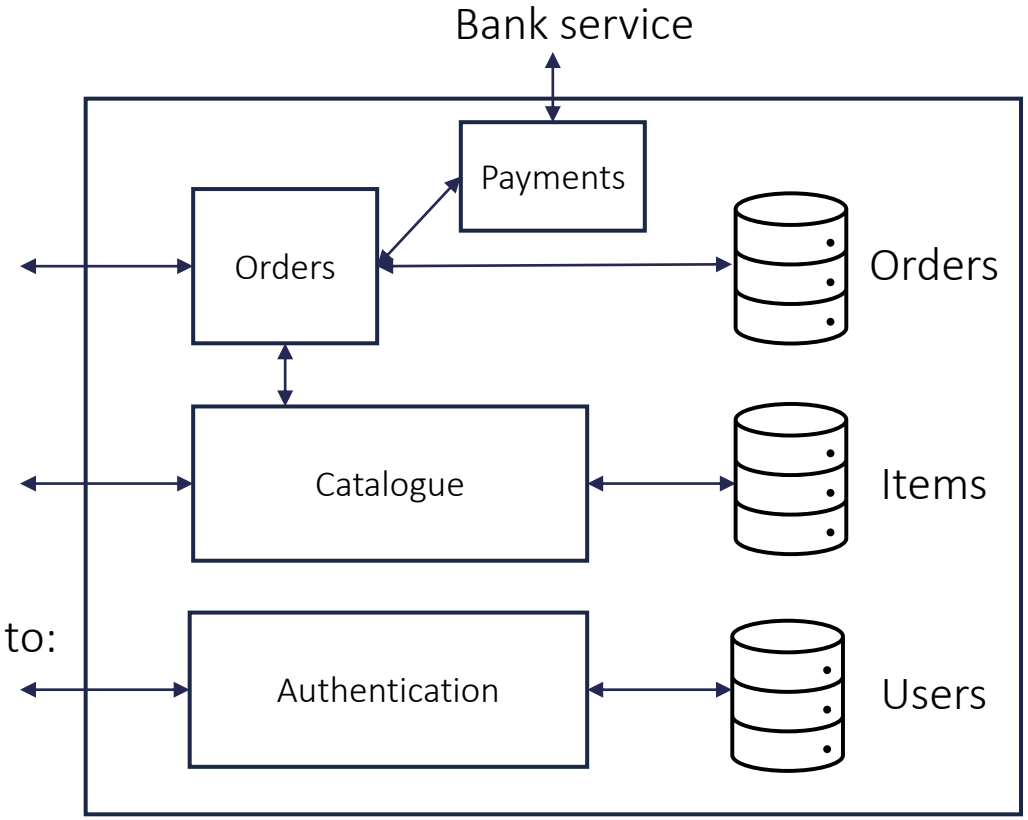
- PUT /orders + JSON payload
- PUT /payment + JSON payload
- GET /orders/{id_oder}

The user access the catalogue to:

- GET /items
- GET /items/{item_id}

The user access the authentication to:

- PUT /login?user={username}…



Orders records

- …

Items records

- Name
- Price
- Stock
- …

Users records

- Username
- Password
- Invoice info
- History
- …

OpenAPI

Example: the architecture

This is an example of the first (optional) delivery we expect.

When drawing the architecture:

- Think of the user stories for the users' operations.
- Think of defining components that have a clear purpose.
- Think that later you will have to code the components and the RESTful API.
- KISS: Keep it Simple and Stupid.



How to deliver the architecture?

- Use a presentation program to create a .pdf file with slides (with the Unipi account you have Office and Google docs for free).
- In the first slide write the name and the members of the group.
- In the second slide put the player information representation.
- In the third slide put the monster collection and the encounter rules.
- In the fourth slide put the complete user stories. (You can use up to 2 slides)
- Use the other slides for the draw of the architecture, the user's interactions and the DB(s).

  You can use more slides than the example but try to stay under 10.
- The files must be uploaded on the course Moodle (you will see how in the project section), only one member of a group need to upload it.



Final recap

Initial main steps:

1. Compose the groups.
2. Choose the monster collection and the encounter rules.
3. Complete the User Stories.
4. Create the architecture.



What happens if we don’t deliver the architecture?

Nothing bad ☺

You lose the chance to have an early feedback on it.

If you are stuck or not sure of something, you can send me an email and/or come to office hours.

